

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	V. Karthik Harish	Reg. No.:	192511328
Date:	06/10/2026	Duration:	60 minutes

Code of Conduct

I V. Karthik Harish (192511328) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *V. Karthik Harish*

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

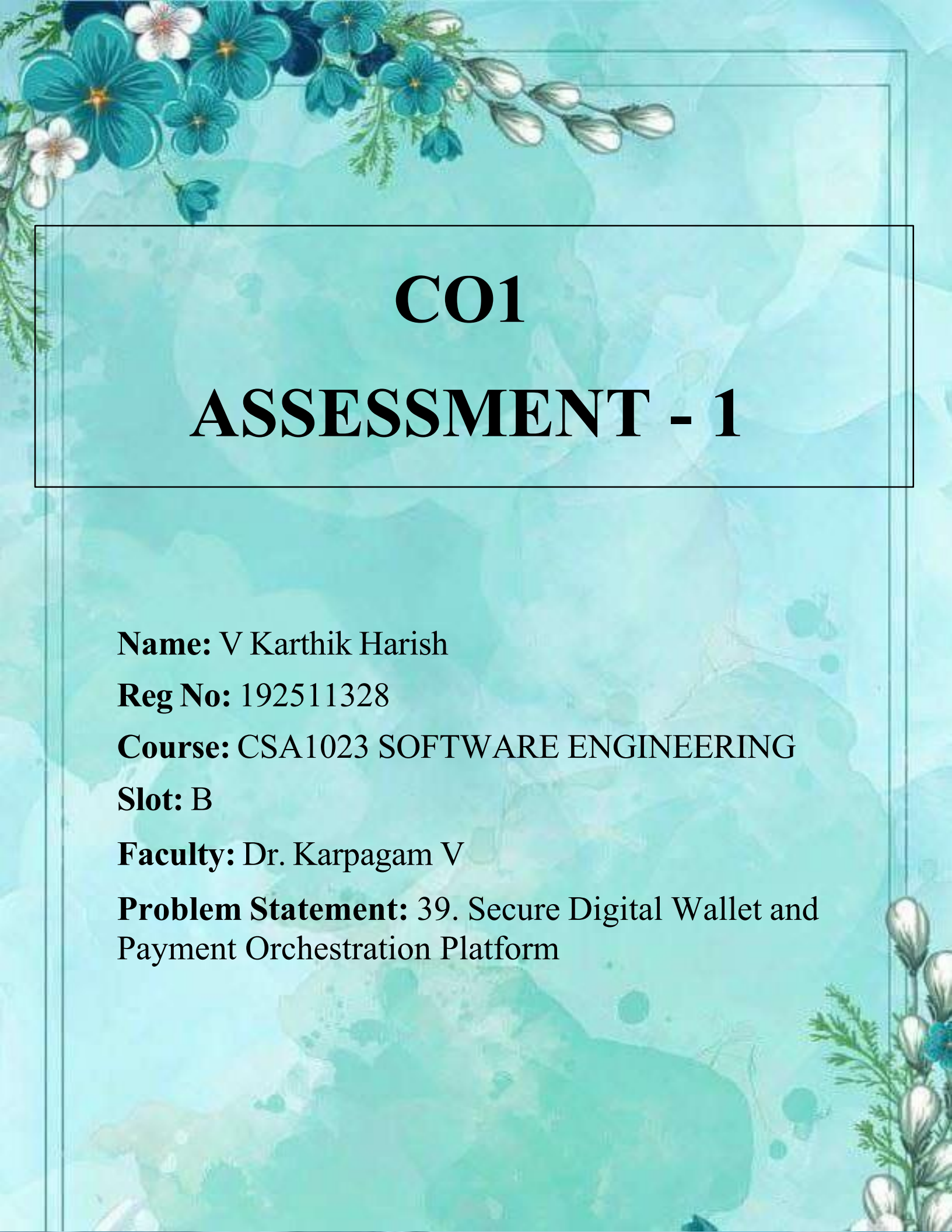
- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.



C01

ASSESSMENT - 1

Name: V Karthik Harish

Reg No: 192511328

Course: CSA1023 SOFTWARE ENGINEERING

Slot: B

Faculty: Dr. Karpagam V

Problem Statement: 39. Secure Digital Wallet and Payment Orchestration Platform

Secure Digital Wallet and Payment Orchestration Platform

1. Understanding the Problem Statement

1.1 The Problem in My Own Words

A digital payment is never one single action. When a customer taps Pay, the system has to verify who the customer is, check the wallet balance or card, talk to an external payment gateway or bank, record the result, and finally tell both the customer and the merchant. Each of these steps is handled by a different service, and any one of them can be slow, fail, or answer twice.

If nothing coordinates these services, the results are familiar and costly: money is debited but the order is not confirmed; a customer is charged twice because they retried a request that timed out; the wallet, the gateway and the ledger each show a different status for the same payment. These inconsistent transaction states lead to financial disputes, refund effort and loss of customer trust.

The problem is therefore to build a **central orchestration layer** that sits between customers, merchants and the payment services. It must (a) drive every transaction through a strictly defined life-cycle, (b) guarantee that a payment request is processed at most once, (c) keep a secure, tamper-evident audit trail, (d) watch itself and raise alerts in real time, and (e) recover automatically when a service fails halfway through a payment.

1.2 Objectives of the Proposed System

#	Objective (from the problem statement)	What the system must do	How success is verified
O1	Process and track digital payment transactions	Accept payment, top-up, transfer and refund requests through secure APIs and expose live status	Every request has a unique transaction ID and queryable status
O2	Maintain consistent transaction states	Enforce a finite state machine; allow only legal transitions	No transaction ever sits in an undefined or contradictory state
O3	Prevent duplicate transaction processing	Idempotency keys on every write API and on every gateway call	Replaying the same request 100 times produces exactly one debit
O4	Integrate multiple payment services securely	Gateway adapters behind one common interface, with signed, encrypted communication	A new gateway can be added without changing core logic
O5	Provide real-time monitoring and alerts	Metrics, structured logs, traces and alert	Failure-rate or latency breach triggers an alert

#	Objective (from the problem statement)	What the system must do	How success is verified
		rules	within minutes
O6	Implement testing, CI/CD and incident recovery	Automated test suites, pipeline gates, retries, reversals and reconciliation	Every commit is tested; stuck transactions self-heal
O7	Maintain secure and auditable transaction records	Append-only ledger and audit log with integrity protection	Any transaction can be reconstructed end-to-end for an auditor

1.3 Scope of the Problem

In scope

- Customer onboarding, authentication and wallet management (balance, top-up, history).
- Payment initiation, authorisation, capture, reversal and refund.
- Orchestration of calls to multiple external payment gateways, banks or UPI/card services through adapters.
- Transaction state management, idempotency, retry, compensation and reconciliation.
- Double-entry ledger and immutable audit trail.
- Monitoring, alerting, logging, automated testing and CI/CD pipeline.
- Operations dashboard for support, finance and security teams.

Out of scope (for this project)

- Building a bank, card network or settlement system; the platform integrates with them, it does not replace them.
- Credit scoring, lending and investment products.
- Native mobile app development (the platform exposes APIs that any client can use).

Assumptions and constraints: external gateways expose REST APIs and webhooks; the platform must follow PCI DSS-style rules by never storing raw card numbers or CVV; applicable financial regulations (for example RBI guidelines in India) and data-protection laws must be respected.

2. Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

Stakeholder	Role in the system	Key responsibilities
Customers (wallet users)	Primary end users	Register, authenticate, top up the wallet, initiate payments, dispute transactions
Merchants	Receive payments	Create payment requests, receive confirmations, request

Stakeholder	Role in the system	Key responsibilities
		refunds, reconcile settlements
Payment gateways, banks and card networks	External service providers	Authorise and settle money movement, send status callbacks and webhooks
Platform operations / SRE team	Run the platform	Monitor health, respond to alerts, handle incidents and recovery
Development, QA and DevOps team	Build and ship the platform	Design, code, test, build pipelines, deploy and maintain the system
Security and compliance officers	Protect and govern	Define security policy, run audits and penetration tests, ensure PCI DSS and data-protection compliance
Finance, reconciliation and dispute-resolution team	Money accuracy	Match ledger with gateway settlements, investigate mismatches, resolve disputes and chargebacks
Regulators and external auditors	Oversight	Demand traceable records, verify compliance and review incident reports
Product owner / business sponsor	Direction and funding	Prioritise features, accept increments, own business value and budget

2.2 Major Challenges Faced by Stakeholders

Stakeholder	Major challenges and issues
Customers	Debited but payment shows failed; duplicate charges after retries; slow refunds; fear of fraud and data theft
Merchants	Unknown payment status at checkout; delayed or mismatched settlements; chargeback handling; abandoned carts due to failures
Gateways and banks	Duplicate requests, timeouts and webhook storms; inconsistent response formats between providers; downtime
Operations team	No single view of transactions; failures discovered late from customer complaints; manual recovery

Stakeholder	Major challenges and issues
	under pressure
Developers and QA	Tight coupling to each gateway; hard-to-test payment flows; risky releases that may break money flows
Security and compliance	Protecting sensitive data across several services; proving who did what and when; keeping up with changing regulation
Finance team	Manual reconciliation; hidden mismatches between wallet, ledger and gateway; dispute evidence scattered across systems

Cross-cutting technical challenges that follow from these:

1. **Partial failure:** a distributed payment touches several services, so it can succeed in one place and fail in another.
2. **Unknown outcomes:** a timeout does not tell us whether the gateway processed the payment; blindly retrying risks a double charge.
3. **Duplicate and out-of-order messages:** clients retry, networks redeliver, and webhooks can arrive before the API response.
4. **Concurrency:** two simultaneous debits on the same wallet must never overdraw it.
5. **Security and auditability** must be built in, not added later.
6. **Scale and safe change:** traffic spikes (festival sales) and frequent releases must not endanger money flows.

3. Analysis of the Current Approach

3.1 The Existing System or Process

The problem statement describes the pain rather than a specific legacy product, so the analysis below describes the **typical current approach** in organisations that have not adopted orchestration. The checkout or wallet application calls each payment service directly (point-to-point integration) using synchronous requests. Each service keeps its own status in its own database. Failed calls are retried with simple, ad-hoc logic, or not at all. Logs are scattered across servers, and a finance team reconciles wallet, bank and gateway statements manually, usually at the end of the day or month. Problems are normally discovered when a customer complains, and recovery is a manual database fix.

3.2 Strengths of the Current Approach

- **Simple to start:** direct integration is quick to build for one gateway and a small user base.
- **Low initial cost:** no extra infrastructure, messaging or orchestration layer to run.
- **Familiar technology:** most teams already know synchronous REST and a relational database.
- **Low latency for the happy path:** fewer hops when everything works.

3.3 Limitations of the Current Approach

- **No single source of truth:** each service holds its own version of the transaction state.

- **No duplicate protection:** retries and double-clicks can cause double debits.
- **Tight coupling:** replacing or adding a gateway means changing business logic.
- **Weak failure handling:** no compensation, so partial failures leave money in limbo.
- **Poor observability:** no end-to-end trace, so a single payment cannot be followed across services.
- **Manual, late reconciliation:** mismatches are found days later and fixed by hand.
- **Weak auditability:** logs are mutable and incomplete, which is a risk in disputes and regulatory audits.
- **Manual, risky releases:** little automated testing and no pipeline gates for payment-critical code.

3.4 Gap Analysis

Gap in current approach	Impact	Capability needed in the new system
No central transaction state	Contradictory statuses, disputes	Orchestrator with a state machine and a single transaction record
No idempotency	Duplicate charges	Idempotency keys enforced at API and gateway layers
Point-to-point coupling	Slow, risky integration of new gateways	Adapter pattern with a common gateway interface
No compensation or recovery	Funds stuck, manual fixes	Saga with automatic reversal, retry and reconciliation
Scattered logs, reactive support	Late detection of failures	Centralised logging, metrics, tracing and real-time alerts
Mutable records	Weak audit and legal evidence	Append-only double-entry ledger and tamper-evident audit log
Manual testing and releases	Regression bugs in money flows	Automated tests and CI/CD with quality gates

4. Proposed Solution Using Software Engineering Principles

4.1 Solution Overview

I propose a **Secure Digital Wallet and Payment Orchestration Platform** built as a set of loosely coupled, containerised services around a central **Payment Orchestrator**. Clients call one secure API gateway; the orchestrator coordinates the wallet, the ledger and the external gateways, and every step is recorded and monitored.

Layer	Components	Purpose
Client layer	Customer app, merchant app, operations dashboard (React)	Entry points for all users

Layer	Components	Purpose
Edge layer	API gateway: TLS termination, rate limiting, request validation, authentication check	Single secure front door
Core services	Auth service, Wallet service, Payment Orchestrator, Ledger service, Notification service	Business logic, each independently deployable
Integration layer	Gateway adapters (Gateway A, Gateway B, bank or UPI), webhook receiver	Hide provider differences behind one interface
Messaging layer	Message broker (for example RabbitMQ or Kafka) with retry and dead-letter queues	Asynchronous, reliable communication between services
Data layer	Relational database for transactions and ledger, in-memory cache for idempotency keys, append-only audit store	Consistent, durable and auditable storage
Observability layer	Metrics, centralised logs, distributed tracing, alert manager	Real-time monitoring and incident response
DevOps layer	Source control, CI/CD pipeline, container registry, infrastructure as code	Automated build, test and deployment

A suitable reference implementation stack is Python (FastAPI) or Java (Spring Boot) for services, PostgreSQL, Redis, a message broker, Docker, GitHub Actions for CI/CD, and Prometheus with Grafana for monitoring. The design itself is technology-neutral.

4.2 Major Functional Modules

Module	Responsibilities
1. Authentication and Authorisation	Registration, login, OAuth2/JWT tokens, multi-factor authentication for high-value actions, role-based access control for customer, merchant, support and admin roles
2. Wallet Management	Create wallet, balance enquiry, top-up, fund reservation (hold), release and debit, with row-level locking to prevent overdraw
3. Payment Orchestrator	Receives payment requests, creates the transaction, runs the saga across services, and owns the state machine

Module	Responsibilities
4. Transaction State Manager	Stores the current state and full history of each transaction; rejects illegal transitions
5. Idempotency Manager	Stores the idempotency key, request fingerprint and response; returns the original result for duplicates
6. Gateway Integration (Adapters)	Translates the common payment interface into each provider's API, maps errors, verifies webhook signatures
7. Ledger and Audit	Double-entry ledger; append-only, hash-chained audit records of every state change and user action
8. Recovery and Reconciliation	Timeout handling, retries with back-off, automatic reversal, dead-letter processing, scheduled reconciliation against gateway reports
9. Monitoring and Alerting	Collects metrics (success rate, latency, stuck transactions), logs and traces; raises alerts to the operations team
10. Notification	Sends SMS, email or push messages and merchant callbacks about transaction outcomes
11. Admin and Operations Dashboard	Search transactions, view timelines, trigger safe manual recovery, export audit reports

4.3 Transaction State Management

Every transaction follows one controlled life-cycle. Any transition that is not listed is rejected.

Current state	Event	Next state	Action taken
INITIATED	Request validated and user authenticated	VALIDATED	Record transaction and idempotency key
VALIDATED	Funds reserved in wallet	RESERVED	Place hold, write ledger entry
RESERVED	Gateway call sent	PROCESSING	Send request with our unique reference
PROCESSING	Gateway confirms success	SUCCESS	Convert hold to debit, notify customer and merchant
PROCESSING	Gateway declines or errors	FAILED	Release hold (compensation)

Current state	Event	Next state	Action taken
PROCESSING	Timeout or no response	UNKNOWN	Query gateway status, schedule retry or reconciliation
UNKNOWN	Reconciliation finds success or failure	SUCCESS or FAILED	Complete or reverse accordingly
SUCCESS	Refund requested and approved	REFUND_PENDING, then REFUNDED	Reverse ledger entries via gateway refund

4.4 How Consistency, Idempotency and Recovery Are Achieved

- **Idempotency:** every write request carries an Idempotency-Key header. The platform saves the key with a hash of the request and the final response under a unique database constraint. A repeated key returns the stored response without re-processing; the same key with a different payload is rejected. The orchestrator also sends its own unique reference to the gateway so the provider can de-duplicate as well.
- **Saga pattern with compensation:** a payment is a sequence of local transactions (reserve, charge, confirm). If a later step fails, defined compensating actions (release hold, reverse entry) restore consistency instead of relying on one large distributed transaction.
- **Transactional outbox:** the state change and the event to be published are written in the same database transaction, so an event is never lost or sent without the state change.
- **Optimistic locking and row versioning:** prevents two concurrent requests from overdrawing a wallet or overwriting each other's state.
- **Retry, circuit breaker and dead-letter queue:** transient errors are retried with exponential back-off and jitter; a failing gateway is isolated by a circuit breaker; messages that keep failing go to a dead-letter queue for investigation.
- **Scheduled reconciliation:** a periodic job compares our ledger with gateway reports and automatically corrects or flags mismatches.

4.5 Functional Requirements

1. **FR1:** The system shall register and authenticate users and enforce role-based access.
2. **FR2:** The system shall create and manage digital wallets, including top-up and balance enquiry.
3. **FR3:** The system shall accept payment, transfer and refund requests through secure REST APIs.
4. **FR4:** The system shall assign a unique ID to every transaction and track it through defined states.
5. **FR5:** The system shall reject illegal state transitions.
6. **FR6:** The system shall detect and block duplicate requests using idempotency keys.
7. **FR7:** The system shall route payments to multiple gateways through a common adapter interface.
8. **FR8:** The system shall process gateway webhooks after verifying their signatures.
9. **FR9:** The system shall automatically retry, reverse or reconcile failed and unknown transactions.
10. **FR10:** The system shall maintain a double-entry ledger and an append-only audit log.
11. **FR11:** The system shall provide real-time dashboards and send alerts on defined thresholds.
12. **FR12:** The system shall notify customers and merchants of transaction outcomes.
13. **FR13:** The system shall allow authorised staff to search transactions and export audit reports.

4.6 Non-Functional Requirements

The figures below are design targets that would be refined with the product owner.

Category	Requirement	Target
Security	Encryption in transit and at rest; no raw card data stored; tokenisation; MFA; secrets in a vault	TLS 1.2 or higher; AES-256 at rest
Reliability and availability	Survive service and gateway failures without losing or duplicating money	99.9% availability; zero lost or duplicated transactions
Performance	Fast API response for normal payments	95th percentile under 2 seconds excluding gateway time
Scalability	Scale horizontally during traffic peaks	Stateless services; auto-scaling; broker-based load levelling
Consistency and integrity	Strong consistency for balances and ledger	ACID transactions on money data; double-entry balance check
Auditability	Complete, tamper-evident history	Append-only, hash-chained records retained per regulation
Maintainability	Easy to change and extend	Modular services; at least 80% automated test coverage; documented APIs
Usability	Clear, simple flows and error messages for users and operations staff	Plain-language failure reasons; one-screen transaction timeline
Observability	Detect problems quickly	Alerts within minutes; end-to-end trace for every transaction
Compliance	Follow PCI DSS-style controls and applicable financial and data-protection rules	Regular audits and access reviews

4.7 Software Engineering Principles Incorporated

Principle	How it is incorporated in the solution
Modularity	Separate services for authentication, wallet, orchestration, ledger and notification; each has one clear responsibility and a defined interface. Gateway adapters isolate provider-specific code.
Scalability	Stateless services behind a load balancer, containerised for horizontal scaling; asynchronous

Principle	How it is incorporated in the solution
	messaging levels out peaks; database read replicas and partitioning of history tables.
Maintainability	Clean layered code, adapter and strategy patterns, API versioning, infrastructure as code, living documentation, automated regression tests and small, frequent releases.
Reliability	Idempotency, saga compensation, outbox pattern, retries, circuit breakers, dead-letter queues, reconciliation and tested recovery runbooks.
Usability	Consistent API design and error codes for developers; real-time status for customers; one-screen transaction timeline for support staff.
Security	Defence in depth: OAuth2/JWT, MFA, role-based access, input validation, rate limiting, signed webhooks, encryption, tokenisation, least privilege and security testing in the pipeline.
Testability and auditability	Dependency injection and mock gateways for testing; every state change logged in an immutable audit trail.

5. Justification of the Chosen Methodology / Framework

5.1 Selected Framework: Agile Scrum delivered through a DevOps (CI/CD) pipeline

I select **Scrum** as the agile project-management framework, combined with **DevOps practices (continuous integration, continuous delivery, infrastructure as code and continuous monitoring)**. Scrum manages *how the team plans and delivers work*; DevOps manages *how that work is built, tested, released and operated safely*.

5.2 Why This Framework Suits the Platform

Characteristic of the project	How Scrum + DevOps responds
Requirements evolve (new gateways, regulations, fraud patterns)	The product backlog is re-prioritised every sprint; change is expected and welcomed
Risk is concentrated in a few critical areas (idempotency, state machine, security)	Build the riskiest stories first and demonstrate working software every 2 weeks
Many stakeholders with different views	Sprint reviews give merchants, finance and security regular feedback; the product owner represents business value
Money-handling code must be defect-free	Every commit passes automated unit, integration,

Characteristic of the project	How Scrum + DevOps responds
	contract and security tests before merge
Frequent change must not endanger production	CI/CD with quality gates, canary or blue-green releases, feature flags and fast rollback
Reliability and recovery are core objectives	Monitoring, alerting and post-incident reviews feed directly back into the backlog

5.3 Comparison with Alternative Models

Model	Assessment for this project
Waterfall	Needs all requirements fixed upfront and delivers only at the end; unsuitable because gateway integrations and regulations change and integration defects would be found too late.
Spiral	Strong risk analysis, but heavy documentation and long cycles; suited to very large, high-cost systems. Its risk focus is covered in Scrum through the risk register and risk-first backlog ordering.
Plain Scrum without DevOps	Good planning but no automated path to safe production releases; weak for a platform whose operation matters as much as its construction.
Scrum + DevOps (chosen)	Iterative delivery, early risk reduction, continuous testing and monitored releases, which together match the project's needs.

5.4 How It Supports Successful Development and Maintenance

Agile project-management techniques applied

- **Roles:** Product Owner (business sponsor), Scrum Master (team facilitator), Development Team including QA, security and DevOps engineers.
- **Artifacts:** product backlog of user stories with acceptance criteria, sprint backlog, increment, burn-down chart and velocity tracking.
- **Events:** sprint planning, daily stand-up, sprint review, sprint retrospective, backlog refinement.
- **Definition of Done:** code reviewed, unit and integration tests passed, security scan clean, documentation and monitoring added, deployed to staging.
- **Risk register:** reviewed every sprint (see Section 6.3).

Indicative release plan (2-week sprints)

Sprint	Goal	Key deliverables
0	Foundation	Repository, CI pipeline skeleton,

Sprint	Goal	Key deliverables
		architecture decisions, environments, risk register
1	Identity and wallet	Authentication, RBAC, wallet creation and top-up, audit logging foundation
2	Core orchestration	Transaction state machine, idempotency manager, payment API with mock gateway
3	Real integrations	Gateway adapters, webhook handling, ledger entries, saga compensation
4	Resilience	Retries, circuit breaker, dead-letter queue, reconciliation job, recovery tests
5	Observability and dashboard	Metrics, alerts, operations dashboard, notification service
6	Hardening and release	Load test, security test, CD to production with canary release, runbooks

CI/CD pipeline stages: commit, build, static analysis and dependency scan, unit tests, integration tests with mock gateways, container build, deploy to staging, automated end-to-end and contract tests, security scan, manual approval gate, canary release to production, automated health checks, automatic rollback on failure.

Maintenance: the same pipeline and backlog continue after launch. Production monitoring, incident post-mortems and retrospectives generate new backlog items, so fixes, new gateways and regulatory changes flow through the same tested, automated route. Infrastructure as code and living documentation make the system easy for new team members to understand and change.

6. Expected Outcomes and Limitations

6.1 Expected Benefits and How Challenges Are Addressed

Expected outcome	How the solution delivers it
Improved payment reliability	Saga, retries, circuit breakers and reconciliation keep payments consistent even when a service fails
Reduced transaction inconsistencies	Single state machine and one authoritative transaction record; illegal transitions are blocked

Expected outcome	How the solution delivers it
Improved security and traceability	End-to-end encryption, tokenisation, MFA, RBAC, immutable audit log and trace IDs
Reduced operational disputes	No duplicate charges through idempotency; complete evidence timeline for every transaction
Scalable transaction processing	Stateless, containerised services with asynchronous messaging and horizontal scaling
Improved recovery from service failures	Automatic reversal, dead-letter handling, tested runbooks and fast rollback

Stakeholder challenge	Addressed by
Customer charged twice	Idempotency keys at API and gateway level
Debited but payment failed	Reservation with automatic compensation and reconciliation
Merchant unsure of payment status	Real-time status API, webhooks and merchant callbacks
Operations discover issues late	Real-time monitoring, alerts and distributed tracing
Developers tied to one gateway	Adapter interface with pluggable gateways
Finance spends days reconciling	Automated scheduled reconciliation and double-entry ledger
Compliance cannot prove what happened	Append-only, hash-chained audit trail and exportable reports

6.2 Additional Benefits

Faster onboarding of new payment partners, lower support costs, higher merchant and customer trust, shorter release cycles with lower risk, and ready evidence for regulatory audits.

6.3 Potential Risks, Implementation Challenges and Mitigation

Risk or challenge	Likelihood / impact	Mitigation
Complexity of distributed design (saga, outbox, messaging)	Medium / High	Build the thinnest vertical slice first; use proven libraries and patterns; design reviews; team training
Third-party gateway downtime or API changes	High / High	Multiple gateways, circuit breaker with failover routing,

Risk or challenge	Likelihood / impact	Mitigation
		contract tests, sandbox monitoring
Security breach or data leak	Low / Very high	Tokenisation, encryption, least privilege, regular penetration testing, dependency scanning, incident response plan
Regulatory change	Medium / High	Compliance owner in the team; regulatory stories in the backlog; configurable rules
Performance bottleneck under peak load	Medium / Medium	Load testing every release; caching; horizontal scaling; database tuning
Incorrect state-machine or ledger logic	Low / Very high	Property-based and integration tests, reconciliation checks, peer review, canary releases
Scope creep or changing priorities	Medium / Medium	Product owner prioritisation, fixed sprint goals, clear Definition of Done
Skill gaps in payments and security	Medium / Medium	Pair programming, workshops, security champion in the team

6.4 Limitations

- Added operational complexity and infrastructure cost compared with a simple direct integration.
- Asynchronous processing and eventual consistency between services mean some states (for example UNKNOWN) are temporary and must be explained clearly to users.
- The platform depends on the availability and honesty of external gateways; it can contain their failures but not remove them.
- Reconciliation can only be as good as the reports the providers supply.
- Full regulatory certification, such as a formal PCI DSS assessment, requires effort and time beyond the software build itself.
- The project scope excludes settlement, lending and native mobile apps.

6.5 Possible Future Enhancements

1. **AI-based fraud and anomaly detection** using transaction patterns and device signals.
2. **Smart routing** that selects the cheapest or most reliable gateway in real time.
3. **Multi-currency and cross-border payments** with exchange-rate management.
4. **Event sourcing** for even richer history and replay of transaction state.
5. **Self-service dispute portal** and automated chargeback handling.
6. **Biometric and risk-based authentication** for stronger and smoother security.
7. **Predictive monitoring** that forecasts gateway degradation before failures occur.
8. **Open APIs and a developer portal** for third-party merchants and fintech partners.

9. **Chaos engineering drills** to continuously verify recovery mechanisms.

7. Conclusion

Digital payments fail not because individual services are weak but because nothing coordinates them. The proposed Secure Digital Wallet and Payment Orchestration Platform places a single, well-tested orchestration layer at the centre. It uses a transaction state machine, idempotency, the saga and outbox patterns, an immutable ledger and real-time observability to deliver reliable, secure and auditable payments. Building it with Scrum and a DevOps CI/CD pipeline lets the team reduce risk early, adapt to change and keep the platform safe to modify long after the first release.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths,	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are

	weaknesses, and identifies all gaps.			largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25